THE NATIONAL UNIVERSITY OF LESOTHO

Test Lead: Test Plan and M2 Test Report

Role document for Milestone M2 · Lereko Thulo

Team	limo-nul (lab group C)	**Release tag**	m2
Role	**Test lead**	**Member**	Lereko Thulo (202322270)
GitHub	recocipher-beep	**Due**	Sun 11 Oct 2026, 23:59

1 Your role in M2

- Own `tests/`: the one-command unit-test runner, the reference model and the hand-derived expectations of the **8 LIMO test programs** (handout B8).
- Make sure the test programs also work as the built-in examples (`src/examples.js`, `examples/*.limo`).
- Prepare the Ripes cross-check and the usability test so that M5 only needs the results filled in.

2 How to run the tests (one command)

```
node tests/run_tests.js        # or:  npm test
# prints PASS/FAIL for every test and ends with:  88 passed, 0 failed
```

Node.js is the only requirement (download from nodejs.org, accept the defaults). If a test fails the command exits with an error code, so it can also be used before every pull request.

3 M2 test report

Group	What it checks	Passed
A. Assembler: encodings (hand-derived, same as RV32I)	Every instruction encodes to the exact RV32I word (incl. the 5 M1 hand-encoded examples); field split; decode round trip	25 / 25
B. Assembler: syntax features	Comments, labels, offsets, hex/binary/negative immediates, case, aliases, Sesotho register names, apostrophes, spacing	12 / 12
C. Assembler: error messages with line numbers	14 kinds of mistake give the right message on the right line; several errors; 1024-instruction limit	16 / 16
D. Control decoder (ID stage)	Control signals of every instruction class, ALU codes, invalid word	8 / 8
E. Hazard unit: forwarding, load-use stall, branch comparator	Forwarding select (priorities, R0, bubbles), load-use stall (incl. no false stall on immediates), signed branch comparator, CPU reset state	15 / 15
F. The 8 LIMO test programs: results and cycle counts (hand-derived)	8 programs: final registers, memory, instruction count, stalls, taken transfers, cycles; all 20 instructions used; examples/ files in sync	12 / 12
Total		**88 / 88**

Handout B8(i) asks for tests of the assembler, the control decoder and the hazard/forwarding unit: groups A–C, D and E. They already pass in M2; in M3/M4 the same functions are called by the pipeline, so the tests keep protecting them.

4 The 8 test programs: hand-derived expectations

Cycle rule: **cycles = N + 4 + load-use stalls + 2 × (taken jumps/branches followed by another executed instruction)**. The reference model (`tests/ref_model.js`) executes each program one instruction at a time and counts the same events, so it can check the pipeline simulator later without sharing its code.

Program	Executed	Stalls	Taken	Cycles	Formula
1. Sum from 1 to 10 (loop)	42	0	10	**64**	42 + 4 + 0 + 2x9
2. Countdown from 10 to 0	31	0	10	**53**	31 + 4 + 0 + 2x9
3. Memory, load-use stall, mask and shift	11	1	0	**16**	11 + 4 + 1
4. RAW hazards resolved by forwarding	5	0	0	**9**	5 + 4
5. Logic, shifts and immediates	12	0	0	**16**	12 + 4
6. All four branches, taken and not taken	9	0	2	**17**	9 + 4 + 2x2
7. Array loop: store 5 numbers, add them up	39	5	4	**56**	39 + 4 + 5 + 2x4
8. Call and return (tlk / tlkr)	6	0	3	**14**	6 + 4 + 2x2

Program	Key hand-derived results (verified by the tests)
1 sum	R5 = 11, R6 = 1+2+...+10 = 55, R7 = 11. Instructions: 3 + 9 × 4 + 3 = 42. khk is not taken 9 times, taken once.
2 countdown	R5 = 0, R6 = 1. Instructions: 2 + 9 × 3 + 2 = 31.
3 memory	Memory[100] = 20, Memory[104] = 30, R7 = 50, R8 = 50 AND 15 = 2, R9 = 50 << 2 = 200, R10 = 100, R11 = 104. One load-use stall (tsl R6 then kpn).
4 raw	R5 = 3, R6 = 6, R7 = 9, R8 = 6, R9 = 12. Every result is used by the next instruction: forwarded, no stall.
5 logic	12 AND 10 = 8, OR = 14, XOR = 6, 12 << 2 = 48, 12 >> 2 = 3, 10 OR 5 = 15, 12 XOR 255 = 243, 12 AND 6 = 4, 10 − 12 = −2 = 0xFFFFFFFE.
6 branches	khl not taken, khsl taken, khn taken, khk not taken: R10 = 1 + 1000 = 1001 (10 and 100 skipped). 9 instructions, 2 taken.
7 array	Array 3,1,4,1,5 at 200..216; sum = 14 stored at 220; R10 ends at 220, R8 = 5. 39 instructions, 5 load-use stalls (one per loop pass), 4 taken khsl.
8 call	tlk R1 gives R1 = 8; subroutine doubles R10 to 14; tlkr returns to address 8; R11 = 15. 6 instructions, 3 taken transfers (the last, tl END, costs nothing extra).

Cycle diagram of program 3 (the load-use case)

Instruction	1	2	3	4	5	6	7	8
`tsl R6, 0(R11)`	IF	ID	EX	MEM	WB			
`kpn R7, R5, R6`		IF	ID	ID stall	EX fwd B	MEM	WB	
`leh R8, R7, 15`			IF	IF held	ID	EX fwd A	MEM	WB

Orange: the stall (kpn waits in ID, leh waits in IF). Green: the loaded value is forwarded from MEM/WB into EX in cycle 5.

5 Ripes cross-check procedure (B8-iii, finished in M5)

1. In the LIMO site, load the program, press Assemble, open the **Export** tab and press **Copy RV32I**.
2. Open Ripes, create a new **Assembly** program and paste the text; choose the 5-stage RISC-V processor model; assemble.
3. Run to the end. Compare: (a) final registers x5..x31 with LIMO R5..R31, (b) data memory, (c) total cycles.
4. Record each program in the table below. If the cycle counts differ (for example because Ripes uses a different branch-resolution stage or counts the final drain differently) **explain the difference** from Ripes' own pipeline description; do not change our numbers to match without understanding why.

Program	Registers agree?	Memory agrees?	LIMO cycles	Ripes cycles	Explanation if different
1 sum			64		
2 countdown			53		
3 memory			16		
4 raw			9		
5 logic			16		
6 branches			17		
7 array			56		
8 call			14		

6 Usability test plan (B8-iv, M5)

Recruit **at least three first-year students outside CS3520**. Give each the 5-minute tasks below, say nothing about how to do them, and note what they do and where they hesitate.

Task	Success means	Observations / change made
1. Load the "Sum from 1 to 10" example and assemble it	finds the Examples menu and the Assemble button alone	
2. Say what the first instruction does and find its hex code	reads the Machine code table	
3. Make one deliberate typo, read the message, fix it	understands the plain-language message	
4. Save the program and open it again	uses Save / Open	

7 Defence preparation

Likely defence question	Answer you should be able to give
Why does Program 1 take 64 cycles, not 66?	42 instructions + 4 fill + 9 × 2 flushes. The tenth taken transfer is the last instruction: no instruction follows it, so its flush delays nothing.
Show a RAW hazard resolved by forwarding.	Program 4: kpn R6,R5,R5 follows kpnh R5; the value is taken from EX/MEM (ForwardA/B = 2); no stall.
Why does Program 3 need a stall although forwarding exists?	The load data exist only after MEM; kpn needs R6 at the start of its EX in the same cycle, so one bubble is needed, then MEM/WB forwarding.
How do you know the expected values are right?	Derived by hand (this document), then confirmed by the reference model, which does not share code with the pipeline.
What is not tested yet?	The five-stage pipeline itself and the visual views (M3/M4); the Ripes comparison and usability test (M5).